

Zoeskoul Linux Learning Path

Four Smaller Courses for Terminal Skills, Permissions, Bash, and Developer Workflow

Curriculum strategy

Do not ship Linux as one giant course. Ship it as a four-course learning path so students can finish each course, build confidence, and unlock the next practical skill step.

Recommended build order: 1) Terminal Fundamentals, 2) Files/Permissions/Users, 3) Bash Scripting, 4) Linux for Developers.

Course	Best For	Core Promise	Capstone
1. Linux Terminal Fundamentals	Absolute beginners	Move around Linux, create files/folders, view text, and stop fearing the terminal.	Organized Linux workspace
2. Linux Files, Permissions, and Users	Core Linux learners, cybersecurity students	Understand rwx permissions, users, groups, sudo, and permission denied errors.	Secure project folder
3. Bash Scripting for Beginners	Students ready to automate	Turn repeated commands into scripts with variables, loops, conditions, and functions.	Bash utility toolkit
4. Linux for Developers	Programming students and junior developers	Use Git, SSH, curl, packages, local servers, processes, and practical troubleshooting.	Developer workflow lab

Platform design rule: Linux should be taught through terminal verification, workspace tasks, debugging scenarios, and small projects. Multiple-choice quizzes should support the course, not drive it.

Learning Path Structure

Stage	Student Capability	Why it matters
Course 1	Can navigate, create, inspect, organize, and safely delete files.	This removes fear of the terminal and prepares students for real tasks.
Course 2	Can understand and fix permissions without guessing.	This is essential for scripts, servers, cybersecurity, and team projects.
Course 3	Can automate repeated terminal work with Bash.	This turns Linux from a command tool into a productivity tool.
Course 4	Can use Linux in a real developer workflow.	This connects Linux to Git, SSH, APIs, local servers, and deployment basics.

Recommended Exercise Types

- **Terminal verification:** student runs commands; the platform checks files, folders, output, or permissions.
- **Workspace task:** student creates or edits a multi-file project in the IDE/workspace.
- **Debugging task:** student sees a real error such as Permission denied and fixes the cause.
- **Command reading quiz:** student explains what a command will do before running it.
- **Mini project:** student combines commands into a meaningful outcome.
- **Capstone project:** student submits a complete workspace that proves they can use the skill in context.

Avoid this

Do not make the course only command memorization: "What does chmod do?"

Prefer this

"You created deploy.sh, but Linux says Permission denied. Fix the script permissions and run it successfully."

Course 1: Linux Terminal Fundamentals

Slug	linux-terminal-fundamentals
Level	Beginner
Prerequisite	No Linux experience required.
Course promise	Students learn to move around Linux, create files and folders, inspect text, and become comfortable using the terminal without fear.

Learning outcomes

- Explain what the terminal is and why developers use it.
- Navigate with absolute and relative paths.
- Create, copy, move, rename, and remove files and directories safely.
- View file contents with common terminal commands.
- Build a small organized Linux workspace from scratch.

Core commands and concepts

`pwd`, `ls`, `cd`, `mkdir`, `touch`, `cp`, `mv`, `rm`, `cat`, `less`, `head`, `tail`, `clear`, `history`, `tree`

Module outline

Module	Lesson	Topics	Student can...
1	What Linux Is	Linux, distributions, shell, terminal, command prompt, home directory	Identify where commands are typed and what a shell does.
2	Navigation Basics	<code>`pwd`</code> , <code>`ls`</code> , <code>`cd`</code> , <code>`.`</code> and <code>`..`</code> , absolute vs relative paths	Move through folders using both path styles.
3	Files and Directories	<code>`touch`</code> , <code>`mkdir`</code> , naming rules, hidden files	Create a clean lab directory with files and subfolders.
4	Copy, Move, Rename, Delete	<code>`cp`</code> , <code>`mv`</code> , <code>`rm`</code> , recursive deletion, safe habits	Organize messy files into the correct folders.
5	Viewing Text Files	<code>`cat`</code> , <code>`less`</code> , <code>`head`</code> , <code>`tail`</code> , file extensions	Inspect notes and logs without opening a GUI editor.
6	Terminal Confidence	<code>history</code> , <code>clear</code> , tab completion, command mistakes	Recover from common beginner terminal mistakes.

Course 1: Linux Terminal Fundamentals - Projects and Capstone

Projects

Project	Description
File Explorer Challenge	Create `linux-lab/notes`, `linux-lab/projects`, and `linux-lab/backups`, then navigate through each folder.
Student Notes Organizer	Move loose files into `school/notes`, `school/homework`, and `school/code`.
Personal Linux Cheat Sheet	Create `commands.md` with at least 15 commands, examples, and explanations.
Course Capstone: Organized Workspace	Build a complete `linux-lab` workspace and submit a command transcript showing how it was created.

Capstone target workspace

```
linux-lab/  
  notes/  
    commands.md  
  projects/  
    hello.txt  
  backups/  
    commands-backup.md
```

Assessment plan

- Command navigation quiz
- File organization task
- Path reading challenge
- Safe deletion scenario
- Final workspace review

Zoeskoul implementation note

Use terminal-verification exercises heavily. Validate folders and files exist instead of asking only multiple-choice questions.

Course 2: Linux Files, Permissions, and Users

Slug	linux-files-permissions-users
Level	Beginner to Core
Prerequisite	Linux Terminal Fundamentals or equivalent comfort with files and folders.
Course promise	Students learn how Linux protects files, why permission denied happens, and how to fix permissions safely.

Learning outcomes

- Read Linux permission strings such as ``-rwxr-x---``.
- Use symbolic and numeric ``chmod`` correctly.
- Understand owner, group, and others.
- Know when ``sudo`` is appropriate and when it is dangerous.
- Debug common permission denied problems.

Core commands and concepts

`ls -l`, `chmod`, `chown`, `chgrp`, `whoami`, `id`, `groups`, `sudo`, `stat`, `umask`

Module outline

Module	Lesson	Topics	Student can...
1	The Permission Model	owner, group, others; read, write, execute	Explain each part of <code>`-rwxr-x---`</code> .
2	File vs Directory Permissions	read/write/execute meaning changes for directories	Predict what users can do inside a directory.
3	Symbolic chmod	<code>`u+x`</code> , <code>`go-r`</code> , <code>`a+r`</code> , adding/removing permissions	Fix a script that cannot run.
4	Numeric chmod	<code>`755`</code> , <code>`644`</code> , <code>`700`</code> , permission math	Set secure permissions using numbers.
5	Ownership and Groups	<code>`chown`</code> , <code>`chgrp`</code> , user IDs, group IDs	Explain why ownership affects access.
6	Sudo and Root Safety	least privilege, sudo habits, risky commands	Choose which commands should or should not use sudo.
7	Permission Debugging	<code>`stat`</code> , <code>`ls -ld`</code> , parent directory access	Find the exact reason access is denied.

Course 2: Linux Files, Permissions, and Users - Projects and Capstone

Projects

Project	Description
Secure Script Folder	Create `secure-lab/private.txt` and `secure-lab/run.sh` with correct owner-only access.
Permission Debugging Lab	Given broken files, repair only the permissions needed to make tests pass.
User and Group Safety Lab	Explain safe vs unsafe use of `sudo` in practical scenarios.
Course Capstone: Multi-File Permission Fix	Secure a small project folder with scripts, public notes, and private config files.

Capstone target workspace

```
secure-project/  
  README.md      # readable by everyone  
  scripts/run.sh  # executable by owner  
  secrets.env     # owner read/write only  
  shared/        # group collaboration folder
```

Assessment plan

- Permission string quiz
- chmod symbolic task
- chmod numeric task
- sudo safety questions
- Final secure folder audit

Zoeskoul implementation note

Best exercises are debugging tasks. Show a permission denied error, then ask the student to fix the minimum permissions required.

Course 3: Bash Scripting for Beginners

Slug	bash-scripting-beginners
Level	Beginner to Intermediate
Prerequisite	Comfort creating files and running commands in Linux.
Course promise	Students turn repeated terminal commands into reusable scripts for automation, backups, logs, and file organization.

Learning outcomes

- Write and run Bash scripts with a proper shebang.
- Use variables, arguments, conditionals, loops, and functions.
- Read input and check command success with exit codes.
- Automate common developer tasks safely.
- Build small command-line tools from scratch.

Core commands and concepts

bash, echo, read, if, test, [], for, while, case, function, \$?, \$1, \$@, date, basename, dirname

Module outline

Module	Lesson	Topics	Student can...
1	Your First Script	shebang, running scripts, executable bit	Create and run `hello.sh`.
2	Variables and Output	variables, quotes, command substitution	Print a formatted system summary.
3	Arguments and Input	`\$1`, `\$2`, `\$@`, `read`, usage messages	Write a script that accepts a filename.
4	Conditionals	`if`, file tests, string tests, numeric tests	Check if a file exists before copying it.
5	Loops	`for`, `while`, looping over files	Process every `.txt` file in a folder.
6	Functions	reusable blocks, parameters, return status	Refactor repeated script logic into functions.
7	Exit Codes and Safety	`\$?`, `set -e`, validation, dry runs	Make scripts fail safely with clear messages.
8	Automation Projects	timestamps, logs, backups, cleanup scripts	Combine everything into a useful tool.

Course 3: Bash Scripting for Beginners - Projects and Capstone

Projects

Project	Description
System Info Script	Print current user, directory, date, kernel, memory, and disk usage.
Backup Script	Copy selected files into a timestamped backup folder.
Log Cleaner	Move old `.log` files into an archive folder and report what changed.
File Organizer	Sort files by extension into folders automatically.
Course Capstone: Bash Utility Toolkit	Create a folder of useful scripts with README documentation and safe error handling.

Capstone target workspace

```
bash-toolkit/  
  scripts/  
    system-info.sh  
    backup.sh  
    organize-files.sh  
    log-cleaner.sh  
  README.md  
  sample-data/
```

Assessment plan

- Script reading quiz
- Fix broken script task
- Argument handling task
- Loop over files task
- Final automation toolkit review

Zoeskoul implementation note

Use real file/folder workspaces. Do not only ask students to write isolated snippets; let them create scripts and run them against sample files.

Course 4: Linux for Developers

Slug	linux-for-developers
Level	Practical Developer Workflow
Prerequisite	Linux Terminal Fundamentals. Bash basics recommended but not required.
Course promise	Students use Linux like working developers: install tools, use Git, connect with SSH, test APIs with curl, and run local projects.

Learning outcomes

- Install and verify common developer tools.
- Use Git branches, commits, pulls, and pushes from Linux.
- Connect to remote machines with SSH and copy files with SCP.
- Use `curl` to inspect web responses and APIs.
- Run and debug a small local developer project.

Core commands and concepts

apt, git, ssh, scp, curl, wget, python3 -m http.server, ss, ps, top, kill, journalctl basics

Module outline

Module	Lesson	Topics	Student can...
1	Developer Workstation Setup	`apt update`, installing tools, verifying versions	Install and verify `git`, `curl`, `tree`, and an editor.
2	Project Folders and Dotfiles	project layout, `.env`, hidden files, README files	Create a clean project directory.
3	Git on Linux	clone, status, add, commit, branch, merge basics	Commit Linux notes on a feature branch.
4	SSH Basics	SSH key idea, `ssh user@host`, known hosts, key permissions	Connect to a local VM or remote server.
5	Copying Files Remotely	`scp`, recursive copy, common permission errors	Copy a project folder to a remote machine.
6	Curl and Web Testing	HTTP status codes, headers, localhost, ports	Test a local web server from terminal.
7	Processes and Ports	`ps`, `top`, `kill`, `ss -tulpn`, background jobs	Find and stop a process using a port.
8	Developer Troubleshooting	logs, environment variables, path problems	Diagnose why a local tool or server is not working.

Course 4: Linux for Developers - Projects and Capstone

Projects

Project	Description
Developer Tool Setup	Install common tools and create a verification checklist.
Linux Notes Git Repo	Create a repo with `commands.md`, `permissions.md`, and `networking.md`.
Remote Backup with SCP	Copy a local project folder to a remote machine or VM.
Local Web Server Test	Run `python3 -m http.server 8000` and test it with `curl`.
Course Capstone: Developer Workflow Lab	Build a local mini website, commit it with Git, run it locally, curl it, and package it for transfer.

Capstone target workspace

```
dev-workflow-lab/  
  website/  
    index.html  
  notes/  
    commands.md  
  scripts/  
    verify-tools.sh  
  README.md
```

Assessment plan

- Tool verification task
- Git branch task
- SSH/SCP scenario
- Curl response task
- Final developer workflow demonstration

Zoeskoul implementation note

Use workspace projects with multiple files. This course should feel close to real developer work rather than a command list.

Build Plan for Zoeskoul

Priority	Build Item	Reason
1	Ship Course 1 first with terminal verification tasks.	Fastest to build and gives beginners immediate confidence.
2	Add permission debugging checks for Course 2.	Permissions are easier to learn when the platform verifies the actual file mode.
3	Make Course 3 script-based with sample files.	Bash needs real input files, logs, and folders to feel useful.
4	Use multi-file developer workspaces for Course 4.	This lets students practice Git-style structure, scripts, local servers, and README documentation.

Minimum viable release for Course 1

1. Six short modules with one reading, one guided command lab, one independent task, and one quiz each.
2. Three mini projects: File Explorer Challenge, Student Notes Organizer, Personal Linux Cheat Sheet.
3. One final capstone workspace with automated file/folder checks.
4. Clear reset behavior so the workspace can be restored to starter files.

Course quality checklist

- Every lesson should answer: what problem does this command solve?
- Every module should include at least one real workspace task.
- Every project should produce visible files or folders the student can inspect.
- Avoid fake memorization. Prefer realistic errors and fixes.
- Keep commands beginner-safe. Delay destructive commands until students understand paths.